Initialization in Rust with `pin-init`

Working towards a chapter in the RfL book about initialization

Benno Lossin <lossin@kernel.org>

Rust for Linux Core Team

Goals of this Talk

Goals of this Talk

- Give an accessible explanation of **why** we need a library *just* for initialization in Rust.

Goals of this Talk

- Give an accessible explanation of **why** we need a library *just* for initialization in Rust.
- Provide an up-to-date introduction to initialization in Rust with `pin-init`.

Goals of this Talk

- Give an accessible explanation of **why** we need a library *just* for initialization in Rust.
- Provide an up-to-date introduction to initialization in Rust with `pin-init`.
- Obtain information about what helps people understand `pin-init`.

Goals of this Talk

- Give an accessible explanation of **why** we need a library *just* for initialization in Rust.
- Provide an up-to-date introduction to initialization in Rust with `pin-init`.
- Obtain information about what helps people understand `pin-init`.
  - Using this, write a chapter for the RfL book.

Goals of this Talk

- Give an accessible explanation of **why** we need a library *just* for initialization in Rust.
- Provide an up-to-date introduction to initialization in Rust with `pin-init`.
- Obtain information about what helps people understand `pin-init`.
  - Using this, write a chapter for the RfL book.
- Give an outlook on the future of `pin-init` & in-place initialization in Rust.

Goals of this Talk

- Give an accessible explanation of **why** we need a library *just* for initialization in Rust.
- Provide an up-to-date introduction to initialization in Rust with `pin-init`.
- Obtain information about what helps people understand `pin-init`.
  - Using this, write a chapter for the RfL book.
- Give an outlook on the future of `pin-init` & in-place initialization in Rust.

Questions welcome, but please keep discussions for later.

Why is Initialization so complicated in Rust?

What happens in C? vs. What is usually done in Rust?

```c
typedef struct {
    u8 data[1024 * 1024];
} buf;

void buf_init(struct buf *p, u8 val)
{
    memset(buf->data, val, 1024 * 1024);
}
```

What happens in C? vs. What is usually done in Rust?

C

```c
typedef struct {
    u8 data[1024 * 1024];
} buf;

void buf_init(struct buf *p, u8 val)
{
    memset(buf->data, val, 1024 * 1024);
}
```

Rust

```rust
struct Buf {
    data: [u8; 1024 * 1024],
}

impl Buf {
    fn new(val: u8) -> Buf {
        Buf { data: [val; 1024 * 1024] }
    }
}
```

What happens in C? vs. What is usually done in Rust?

```c
typedef struct {
    u8 data[1024 * 1024];
} buf;

void buf_init(struct buf *p, u8 val)
{
    memset(buf->data, val, 1024 * 1024);
}
```




```rust
struct Buf {
    data: [u8; 1024 * 1024],
}

impl Buf {
    fn new(val: u8) -> Buf {
        Buf { data: [val; 1024 * 1024] }
    }
}
```

What happens in C? vs. What is usually done in Rust?

C

```c
typedef struct {
    u8 data[1024 * 1024];
} buf;

void buf_init(struct buf *p, u8 val)
{
    memset(buf->data, val, 1024 * 1024);
}
```

out-pointer

Rust

```rust
struct Buf {
    data: [u8; 1024 * 1024],
}

impl Buf {
    fn new(val: u8) -> Buf {
        Buf { data: [val; 1024 * 1024] }
    }
}
```

by-value

What happens in C? vs. What is usually done in Rust?

C

```c
typedef struct {
    u8 data[1024 * 1024];
} buf;

void buf_init(struct buf *p, u8 val)
{
    memset(buf->data, val, 1024 * 1024);
}
```

out-pointer

Rust

```rust
struct Buf {
    data: [u8; 1024 * 1024],
}

impl Buf {
    fn new(val: u8) -> Buf {
        Buf { data: [val; 1024 * 1024] }
    }
}
```

by-value

overflows the stack!

Out-Pointer *Conventions* in C

```c
void buf_init(struct buf *p, u8 val)
{
    memset(buf->data, val, 1024 * 1024);
}
```

Out-Pointer *Conventions* in C

```C
void buf_init(struct buf *p, u8 val)
{
    memset(buf->data, val, 1024 * 1024);
}
```

- Using out-pointers requires the function to *always* initialize the out-pointer value.

Out-Pointer *Conventions* in C

```c
void buf_init(struct buf *p, u8 val)
{
    memset(buf->data, val, 1024 * 1024);
}
```

- Using out-pointers requires the function to *always* initialize the out-pointer value.
- Except when they support errors. In the error path, no data should be stored in the out-pointer that needs to be cleaned up.

Out-Pointer *Conventions* in C

```c
void buf_init(struct buf *p, u8 val)
{
    memset(buf->data, val, 1024 * 1024);
}
```

- Using out-pointers requires the function to *always* initialize the out-pointer value.
- Except when they support errors. In the error path, no data should be stored in the out-pointer that needs to be cleaned up.
- Except...

Out-Pointer *Conventions* in C

Can be broken in some cases!

```C
void buf_init(struct buf *p, u8 val)
{
    memset(buf->data, val, 1024 * 1024);
}
```

- Using out-pointers requires the function to *always* initialize the out-pointer value.
- Except when they support errors. In the error path, no data should be stored in the out-pointer that needs to be cleaned up.
- Except...

Out-Pointer *Conventions* in C

Can be broken in some cases!

```C
void buf_init(struct buf *p, u8 val)
{
    memset(buf->data, val, 1024 * 1024);
}
```

- Using out-pointers requires the function to *always* initialize the out-pointer value.
- Except when they support errors. In the error path, no data should be stored in the out-pointer that needs to be cleaned up.
- Except...

Inherently in conflict with safe Rust:

All safely accessible data in Rust must be initialized.

Out-Pointer *Conventions* in C

Can be broken in some cases!

```C
1 void buf_init(struct buf *p, u8 val)
2 {
3     memset(buf->data, val, 1024 * 1024);
4 }
```

- Using out-pointers requires the function to *always* initialize the out-pointer value.
- Except when they support errors. In the error path, no data should be stored in the out-pointer that needs to be cleaned up.
- Except...

Inherently in conflict with safe Rust:

Cannot be broken!

All safely accessible data in Rust must be initialized.

Out-pointers in Rust

- Out-pointers don't exist in Rust, they would have to:

Out-pointers in Rust

- Out-pointers don't exist in Rust, they would have to:
  - *Somehow* ensure initialization before returning from the function,

Out-pointers in Rust

- Out-pointers don't exist in Rust, they would have to:
  - *Somehow* ensure initialization before returning from the function,
  - *Except* in the error path, where they ensure that they carry no initialized data.

Out-pointers in Rust

- Out-pointers don't exist in Rust, they would have to:
  - *Somehow* ensure initialization before returning from the function,
  - *Except* in the error path, where they ensure that they carry no initialized data.

This is *difficult*!

A more advanced Example in C

```c
typedef struct {
    struct mutex lock;
    struct buf buffer;
} my_driver_data;
```

A more advanced Example in C

```C
typedef struct {
    struct mutex lock;
    struct buf buffer;
} my_driver_data;

int my_driver_probe(struct foo_device *dev)
{
    dev->private = kmalloc(sizeof(struct my_driver_data), GFP_KERNEL);
    if (!dev->private) {
        return -ENOMEM;
    }



}
```

A more advanced Example in C

```c
typedef struct {
    struct mutex lock;
    struct buf buffer;
} my_driver_data;

int my_driver_probe(struct foo_device *dev)
{
    dev->private = kmalloc(sizeof(struct my_driver_data), GFP_KERNEL);
    if (!dev->private) {
        return -ENOMEM;
    }
    struct my_driver_data *ptr = (struct my_driver_data *) dev->private;


    return 0;
}
```

A more advanced Example in C

```c
typedef struct {
    struct mutex lock;
    struct buf buffer;
} my_driver_data;

int my_driver_probe(struct foo_device *dev)
{
    dev->private = kmalloc(sizeof(struct my_driver_data), GFP_KERNEL);
    if (!dev->private) {
        return -ENOMEM;
    }
    struct my_driver_data *ptr = (struct my_driver_data *) dev->private;
    mutex_init(&ptr->lock);

    return 0;
}
```

A more advanced Example in C

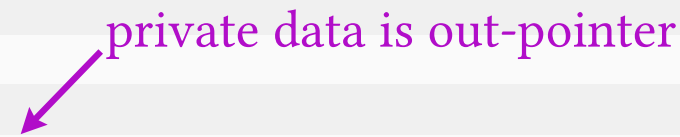
```C
typedef struct {
    struct mutex lock;
    struct buf buffer;
} my_driver_data;

int my_driver_probe(struct foo_device *dev)
{
    dev->private = kmalloc(sizeof(struct my_driver_data), GFP_KERNEL);
    if (!dev->private) {
        return -ENOMEM;
    }
    struct my_driver_data *ptr = (struct my_driver_data *) dev->private;
    mutex_init(&ptr->lock);
    buf_init(&ptr->buffer, 0);
    return 0;
}
```

A more advanced Example in C

```c
typedef struct {
    struct mutex lock;
    struct buf buffer;
} my_driver_data;

int my_driver_probe(struct foo_device *dev)
{
    dev->private = kmalloc(sizeof(struct my_driver_data), GFP_KERNEL);
    if (!dev->private) {
        return -ENOMEM;
    }
    struct my_driver_data *ptr = (struct my_driver_data *) dev->private;
    mutex_init(&ptr->lock);
    buf_init(&ptr->buffer, 0);
    return 0;
}
```



A mutex must not be mem-copied after initializing it.

Implicit Convention in C

A mutex must not be moved after initializing it.

A mutex must not be moved after initializing it.

- In Rust, we say that we *pin* a value when we require it to never move again.

A mutex must not be moved after initializing it.

- In Rust, we say that we *pin* a value when we require it to never move again.
- Thus, a mutex is *pinned* at creation.

How does Initialization look like with `pin-init`?

A Basic Example of `pin-init`

Rust

```rust
struct Buf {
    data: [u8; 1024 * 1024],
}

impl Buf {
    fn new(val: u8) -> Self {
        Self {
            data: [val; 1024 * 1024],
        }
    }
}
```

A Basic Example of `pin-init`

Rust

```rust
struct Buf {
    data: [u8; 1024 * 1024],
}

impl Buf {
    fn new(val: u8) -> impl Init<Self> {
        Self {
            data: [val; 1024 * 1024],
        }
    }
}
```

A Basic Example of `pin-init`

Rust

```rust
struct Buf {
    data: [u8; 1024 * 1024],
}

impl Buf {
    fn new(val: u8) -> impl Init<Self> {
        init!(Self {
            data: [val; 1024 * 1024],
        })
    }
}
```

A Basic Example of `pin-init`

Rust

```rust
struct Buf {
    data: [u8; 1024 * 1024],
}

impl Buf {
    fn new(val: u8) -> impl Init<Self> {
        init!(Self {
            data <- init_array_from_fn::<_, 1024 * 1024, _, _>(|_index| val),
        })
    }
}
```

A Basic Example of `pin-init`

```rust
struct Buf {
    data: [u8; 1024 * 1024],
}

impl Buf {
    fn new(val: u8) -> impl Init<Self> {
        init!(Self {
            data <- init_array_from_fn::<_, 1024 * 1024, _, _>(|_index| val),
        })
    }
}
```

Signifies in-place initialization

A Basic Example of `pin-init`

```rust
struct Buf {
    data: [u8; 1024 * 1024],
}

impl Buf {
    fn new(val: u8) -> impl Init<Self> {
        init!(Self {
            data <- init_array_from_fn(|_| val),
        })
    }
}
```

A Basic Example of `pin-init`

Rust

```rust
struct Buf {
    data: [u8; 1024 * 1024],
    count: usize,
}

impl Buf {
    fn new(val: u8) -> impl Init<Self> {
        init!(Self {
            data <- init_array_from_fn(|_| val),
            count: 42,
        })
    }
}
```

The Details of `pin-init`

- `Init<T>` trait is an *initializer:*

The Details of `pin-init`

- `Init<T>` trait is an *initializer:*
  - It is an object that carries the data needed to initialize a `T`.

The Details of `pin-init`

- `Init<T>` trait is an *initializer:*
  - It is an object that carries the data needed to initialize a `T`.
  - When it isn't run, no data is initialized.

The Details of `pin-init`

- `Init<T>` trait is an *initializer:*
  - It is an object that carries the data needed to initialize a `T`.
  - When it isn't run, no data is initialized.
- Several ways to create an initializer:

The Details of `pin-init`

- `Init<T>` trait is an *initializer:*
  - It is an object that carries the data needed to initialize a `T`.
  - When it isn't run, no data is initialized.
- Several ways to create an initializer:
  - Any value `T` can be turned into an `Init<T>`.

The Details of `pin-init`

- `Init<T>` trait is an *initializer:*
  - It is an object that carries the data needed to initialize a `T`.
  - When it isn't run, no data is initialized.
- Several ways to create an initializer:
  - Any value `T` can be turned into an `Init<T>`.
  - The `init!` macro creates an initializer for a struct given initializers for each field.

The Details of `pin-init`

- `Init<T>` trait is an *initializer:*
  - It is an object that carries the data needed to initialize a `T`.
  - When it isn't run, no data is initialized.
- Several ways to create an initializer:
  - Any value `T` can be turned into an `Init<T>`.
  - The `init!` macro creates an initializer for a struct given initializers for each field.
  - Several functions in `pin-init` return initializers (for example `zeroed` for zeroable data).

The Details of `pin-init`

- `Init<T>` trait is an *initializer:*
  - It is an object that carries the data needed to initialize a `T`.
  - When it isn't run, no data is initialized.
- Several ways to create an initializer:
  - Any value `T` can be turned into an `Init<T>`.
  - The `init!` macro creates an initializer for a struct given initializers for each field.
  - Several functions in `pin-init` return initializers (for example `zeroed` for zeroable data).
- Need all of this again with slight variations for `pin`:
  - `PinInit` trait, which requires the value to be *pinned* after initialization.
  - `pin_init!` macro, which allows composing `PinInit` for structs.

The Details of `pin-init`

- `Init<T>` trait is an *initializer:*
  - It is an object that carries the data needed to initialize a `T`.
  - When it isn't run, no data is initialized.
- Several ways to create an initializer:
  - Any value `T` can be turned into an `Init<T>`.
  - The `init!` macro creates an initializer for a struct given initializers for each field.
  - Several functions in `pin-init` return initializers (for example `zeroed` for zeroable data).
- Need all of this again with slight variations for `pin`:
  - `PinInit` trait, which requires the value to be *pinned* after initialization.
  - `pin_init!` macro, which allows composing `PinInit` for structs.
  - *Additionally:* compatibility with `Init`, since any initializer also is a pin-initializer.

Converting the `Mutex` Example

```c
typedef struct {
    struct mutex lock;
    struct buf buffer;
} my_driver_data;

int my_driver_probe(struct foo_device *dev)
{
    dev->private = kmalloc(sizeof(struct my_driver_data), GFP_KERNEL);
    if (!dev->private) {
        return -ENOMEM;
    }
    struct my_driver_data *ptr = (struct my_driver_data *) dev->private;
    mutex_init(&ptr->lock);
    buf_init(&ptr->buffer, 0);
    return 0;
}
```

Converting the `Mutex` Example

```rust
struct MyDriverData {
    buffer: Mutex<Buf>,
}
```

Rust

```c
int my_driver_probe(struct foo_device *dev)
{
    dev->private = kmalloc(sizeof(struct my_driver_data), GFP_KERNEL);
    if (!dev->private) {
        return -ENOMEM;
    }
    struct my_driver_data *ptr = (struct my_driver_data *) dev->private;
    mutex_init(&ptr->lock);
    buf_init(&ptr->buffer, 0);
    return 0;
}
```

C

Converting the `Mutex` Example

```rust
struct MyDriverData {
    buffer: Mutex<Buf>,
}
```

Rust

combines `buffer` and `lock` into one field

```c
int my_driver_probe(struct foo_device *dev)
{
    dev->private = kmalloc(sizeof(struct my_driver_data), GFP_KERNEL);
    if (!dev->private) {
        return -ENOMEM;
    }
    struct my_driver_data *ptr = (struct my_driver_data *) dev->private;
    mutex_init(&ptr->lock);
    buf_init(&ptr->buffer, 0);
    return 0;
}
```

C

Converting the `Mutex` Example

Rust

```rust
struct MyDriverData {
    buffer: Mutex<Buf>,
}

impl FooDriver for MyDriver {
    type Data = MyDriverData;

    fn probe(dev: ARef<FooDevice>) -> impl PinInit<Self::Data> {
```

C

```c
        dev->private = kmalloc(sizeof(struct my_driver_data), GFP_KERNEL);
        if (!dev->private) {
            return -ENOMEM;
        }
        struct my_driver_data *ptr = (struct my_driver_data *) dev->private;
        mutex_init(&ptr->lock);
        buf_init(&ptr->buffer, 0);
        return 0;
    }
}
```

Converting the `Mutex` Example

```rust
struct MyDriverData {
    buffer: Mutex<Buf>,
}

impl FooDriver for MyDriver {
    type Data = MyDriverData;

    fn probe(dev: ARef<FooDevice>) -> impl PinInit<Self::Data> {
```

Rust

Ref-counted objects in Rust use the `ARef` pointer

```c
        dev->private = kmalloc(sizeof(struct my_driver_data), GFP_KERNEL);
        if (!dev->private) {
            return -ENOMEM;
        }
        struct my_driver_data *ptr = (struct my_driver_data *) dev->private;
        mutex_init(&ptr->lock);
        buf_init(&ptr->buffer, 0);
        return 0;
    }
}
```

C

Converting the `Mutex` Example

```rust
struct MyDriverData {
    buffer: Mutex<Buf>,
}

impl FooDriver for MyDriver {
    type Data = MyDriverData;

    fn probe(dev: ARef<FooDevice>) -> impl PinInit<Self::Data> {
```

Rust

Ref-counted objects in Rust use the `ARef` pointer

We return the private data directly

```c
        dev->private = kmalloc(sizeof(struct my_driver_data), GFP_KERNEL);
        if (!dev->private) {
            return -ENOMEM;
        }
        struct my_driver_data *ptr = (struct my_driver_data *) dev->private;
        mutex_init(&ptr->lock);
        buf_init(&ptr->buffer, 0);
        return 0;
    }
}
```

C

Converting the Mutex Example

Rust

```rust
struct MyDriverData {
    buffer: Mutex<Buf>,
}

impl FooDriver for MyDriver {
    type Data = MyDriverData;

    fn probe(dev: ARef<FooDevice>) -> impl PinInit<Self::Data> {
```

Ref-counted objects in Rust use the `ARef` pointer

We return the private data directly

C

```c


        struct my_driver_data *ptr = (struct my_driver_data *) dev->private;
        mutex_init(&ptr->lock);
        buf_init(&ptr->buffer, 0);
        return 0;
    }
}
```

Converting the `Mutex` Example

Rust

```rust
struct MyDriverData {
    buffer: Mutex<Buf>,
}

impl FooDriver for MyDriver {
    type Data = MyDriverData;

    fn probe(dev: ARef<FooDevice>) -> impl PinInit<Self::Data> {
```

Ref-counted objects in Rust use the `ARef` pointer

We return the private data directly

C

```c


        mutex_init(&ptr->lock); // Mutex::new(...)
        buf_init(&ptr->buffer, 0); // Buf::new(0)

    }
}
```

Converting the `Mutex` Example

```rust
struct MyDriverData {
    buffer: Mutex<Buf>,
}

impl FooDriver for MyDriver {
    type Data = MyDriverData;

    fn probe(dev: ARef<FooDevice>) -> impl PinInit<Self::Data> {
```

Ref-counted objects in Rust use the `ARef` pointer

We return the private data directly

```rust

        pin_init!(MyDriverData {
            buffer <- Mutex::new(Buf::new(0)),
        })
    }
}
```

Converting the `Mutex` Example

Rust

```rust
struct MyDriverData {

    buffer: Mutex<Buf>,
}

impl FooDriver for MyDriver {
    type Data = MyDriverData;

    fn probe(dev: ARef<FooDevice>) -> impl PinInit<Self::Data> {
        pin_init!(MyDriverData {
            buffer <- Mutex::new(Buf::new(0)),
        })
    }
}
```

Converting the `Mutex` Example

Rust

```rust
#[pin_data]
struct MyDriverData {
    #[pin]
    buffer: Mutex<Buf>,
}

impl FooDriver for MyDriver {
    type Data = MyDriverData;

    fn probe(dev: ARef<FooDevice>) -> impl PinInit<Self::Data> {
        pin_init!(MyDriverData {
            buffer <- Mutex::new(Buf::new(0)),
        })
    }
}
```

Converting the `Mutex` Example

```rust
#[pin_data] ← Required for using pin_init! with this struct
struct MyDriverData {
    #[pin]
    buffer: Mutex<Buf>,
}

impl FooDriver for MyDriver {
    type Data = MyDriverData;

    fn probe(dev: ARef<FooDevice>) -> impl PinInit<Self::Data> {
        pin_init!(MyDriverData {
            buffer <- Mutex::new(Buf::new(0)),
        })
    }
}
```

Rust

Converting the `Mutex` Example

```rust
#[pin_data] ← Required for using pin_init! with this struct
struct MyDriverData {
    #[pin] ← Marks the buffer field as structurally pinned
    buffer: Mutex<Buf>,
}

impl FooDriver for MyDriver {
    type Data = MyDriverData;

    fn probe(dev: ARef<FooDevice>) -> impl PinInit<Self::Data> {
        pin_init!(MyDriverData {
            buffer <- Mutex::new(Buf::new(0)),
        })
    }
}
```

Error Handling

Error Handling

Rust

```rust
#[pin_data]
struct MyDriverData {
    shared_data: Arc<SharedData>,
    #[pin]
    buffer: Mutex<Buf>,
}

impl MyDriverData {
     fn new() -> impl PinInit<Self> {
         pin_init!(MyDriverData {


         })
    }
}
```

Error Handling

```rust
#[pin_data]
struct MyDriverData {
    shared_data: Arc<SharedData>,
    #[pin]
    buffer: Mutex<Buf>,
}

impl MyDriverData {
     fn new() -> impl PinInit<Self> {
         pin_init!(MyDriverData {


         })
    }
}
```

Arc creation is fallible, since it allocates

Error Handling

```rust
#[pin_data]
struct MyDriverData {
    shared_data: Arc<SharedData>,
    #[pin]
    buffer: Mutex<Buf>,
}

impl MyDriverData {
    fn new() -> impl PinInit<Self, Error> {
        pin_init!(MyDriverData {


        })
    }
}
```

Arc creation is fallible, since it allocates

Error Handling

```rust
#[pin_data]
struct MyDriverData {
    shared_data: Arc<SharedData>,
    #[pin]
    buffer: Mutex<Buf>,
}

impl MyDriverData {
    fn new() -> impl PinInit<Self, Error> {
        pin_init!(MyDriverData {

            buffer <- Mutex::new(Buf::new(0)),
        })
    }
}
```

Arc creation is fallible, since it allocates

Error Handling

```rust
#[pin_data]
struct MyDriverData {
    shared_data: Arc<SharedData>,
    #[pin]
    buffer: Mutex<Buf>,
}

impl MyDriverData {
    fn new() -> impl PinInit<Self, Error> {
        pin_init!(MyDriverData {
            shared_data: Arc::new(SharedData::new(), GFP_KERNEL)?,
            buffer <- Mutex::new(Buf::new(0)),
        })
    }
}
```

Arc creation is fallible, since it allocates

Error Handling

```rust
#[pin_data]
struct MyDriverData {
    shared_data: Arc<SharedData>,
    #[pin]
    buffer: Mutex<Buf>,
}

impl MyDriverData {
    fn new() -> impl PinInit<Self, Error> {
        pin_init!(MyDriverData {
            shared_data: Arc::new(SharedData::new(), GFP_KERNEL)?,
            buffer <- Mutex::new(Buf::new(0)),
        })
    }
}
```

Arc creation is fallible, since it allocates

Can use ? operator which short-circuits on error

New Features and Language Integration

New Features

New features added over the last two years:

New Features

New features added over the last two years:

- Code blocks to run arbitrary code before, between, and after initializing fields,

New Features

New features added over the last two years:

- Code blocks to run arbitrary code before, between, and after initializing fields,
- Access to previously initialized fields in code blocks and later fields,

New Features

New features added over the last two years:

- Code blocks to run arbitrary code before, between, and after initializing fields,
- Access to previously initialized fields in code blocks and later fields,
- Pin projections to convert `Pin<&mut Struct>` into `Pin<&mut Field>` or `&mut Field`,

New Features

New features added over the last two years:

- Code blocks to run arbitrary code before, between, and after initializing fields,
- Access to previously initialized fields in code blocks and later fields,
- Pin projections to convert `Pin<&mut Struct>` into `Pin<&mut Field>` or `&mut Field`,
- `[pin_]init_scope` function to compute data before creating the initializer and pass it to multiple fields.

Feature: Code Blocks & Access to Initialized Fields

```rust
struct MyStruct {
    field1: i32,
    field2: u32,
}

fn foo();
fn bar(x: &mut i32);
fn baz(x: &mut i32, y: &mut u32);
fn bam();
```

```rust
init!(MyStruct {

    field1: 42,


    field2: 24,


})
```

Feature: Code Blocks & Access to Initialized Fields

```rust
struct MyStruct {
    field1: i32,
    field2: u32,
}

fn foo();
fn bar(x: &mut i32);
fn baz(x: &mut i32, y: &mut u32);
fn bam();
```

```rust
init!(MyStruct {
    _: {
        foo();
    },
    field1: 42,



    field2: 24,


})
```

Feature: Code Blocks & Access to Initialized Fields

```rust
struct MyStruct {
    field1: i32,
    field2: u32,
}

fn foo();
fn bar(x: &mut i32);
fn baz(x: &mut i32, y: &mut u32);
fn bam();
```

```rust
init!(MyStruct {
    _: {
        foo();
    },
    field1: 42,
    _: {
        foo();
        bar(field1);
    },
    field2: 24,
})
```

Feature: Code Blocks & Access to Initialized Fields

```rust
struct MyStruct {
    field1: i32,
    field2: u32,
}

fn foo();
fn bar(x: &mut i32);
fn baz(x: &mut i32, y: &mut u32);
fn bam();
```

```rust
init!(MyStruct {
    _: {
        foo();
    },
    field1: 42,
    _: {
        foo();
        bar(field1);
    },
    field2: 24,
    _: {
        baz(field1, field2);
    },


})
```

Feature: Code Blocks & Access to Initialized Fields

```rust
struct MyStruct {
    field1: i32,
    field2: u32,
}

fn foo();
fn bar(x: &mut i32);
fn baz(x: &mut i32, y: &mut u32);
fn bam();
```

```rust
init!(MyStruct {
    _: {
        foo();
    },
    field1: 42,
    _: {
        foo();
        bar(field1);
    },
    field2: 24,
    _: {
        baz(field1, field2);
    },
    _: {
        bam();
    },
})
```

init_scope

- Sometimes one needs to pass data to multiple fields at the same time,

`init_scope`

- Sometimes one needs to pass data to multiple fields at the same time,
- `fn [pin_]init_scope(...)` serves this purpose:

init_scope

- Sometimes one needs to pass data to multiple fields at the same time,
- `fn [pin_]init_scope(...)` serves this purpose:

```rust
init_scope(|| {
    let foo = Foo::new();
    init!(Bar {
        bar: foo.bar.clone(),
        foo,
    })
})
```

Pin Projections

```rust
#[pin_data]
struct Struct {
    field1: usize,
    #[pin]
    field2: Mutex<usize>,
}
```

Pin Projections

```rust
#[pin_data]
struct Struct {
    field1: usize,
    #[pin]
    field2: Mutex<usize>,
}
```

Auto-generated by `#[pin_data]`:

```rust
struct PinProjectedStruct<'a> {
    field1: &'a mut usize,
    field2: Pin<&'a mut Mutex<usize>>,
}

impl Struct {
    fn project(self: Pin<&mut Self>) -> PinProjectedStruct<'_> {
        // Uses unsafe code to perform the correct projections.
    }
}
```

Upcoming Features of `pin-init`

Upcoming Features of `pin-init`

- Many improvements coming to `pin-init` due to more usage (especially in DRM & by Danilo, many thanks!).

Upcoming Features of `pin-init`

- Many improvements coming to `pin-init` due to more usage (especially in DRM & by Danilo, many thanks!).
  - This is great, keep the suggestions coming!

Upcoming Features of `pin-init`

- Many improvements coming to `pin-init` due to more usage (especially in DRM & by Danilo, many thanks!).
  - This is great, keep the suggestions coming!
- Big improvement *behind the scenes:* `syn` to parse Rust code instead of declarative macros. This allows for:

Upcoming Features of `pin-init`

- Many improvements coming to `pin-init` due to more usage (especially in DRM & by Danilo, many thanks!).
  - This is great, keep the suggestions coming!
- Big improvement *behind the scenes:* `syn` to parse Rust code instead of declarative macros. This allows for:
  - Better error reporting and better syntax support,

Upcoming Features of `pin-init`

- Many improvements coming to `pin-init` due to more usage (especially in DRM & by Danilo, many thanks!).
  - This is great, keep the suggestions coming!
- Big improvement *behind the scenes:* `syn` to parse Rust code instead of declarative macros. This allows for:
  - Better error reporting and better syntax support,
  - Comfortable maintenance (Gary Guo will be co-maintaining the library when it switches to `syn`),

Upcoming Features of `pin-init`

- Many improvements coming to `pin-init` due to more usage (especially in DRM & by Danilo, many thanks!).
  - This is great, keep the suggestions coming!
- Big improvement *behind the scenes:* `syn` to parse Rust code instead of declarative macros. This allows for:
  - Better error reporting and better syntax support,
  - Comfortable maintenance (Gary Guo will be co-maintaining the library when it switches to `syn`),
  - Easier extensions of the existing syntax.

Upcoming Features of `pin-init`

- Many improvements coming to `pin-init` due to more usage (especially in DRM & by Danilo, many thanks!).
  - This is great, keep the suggestions coming!
- Big improvement *behind the scenes:* `syn` to parse Rust code instead of declarative macros. This allows for:
  - Better error reporting and better syntax support,
  - Comfortable maintenance (Gary Guo will be co-maintaining the library when it switches to `syn`),
  - Easier extensions of the existing syntax.
- Closure syntax that improves upon `init_scope`,

Upcoming Features of `pin-init`

- Many improvements coming to `pin-init` due to more usage (especially in DRM & by Danilo, many thanks!).
  - This is great, keep the suggestions coming!
- Big improvement *behind the scenes:* `syn` to parse Rust code instead of declarative macros. This allows for:
  - Better error reporting and better syntax support,
  - Comfortable maintenance (Gary Guo will be co-maintaining the library when it switches to `syn`),
  - Easier extensions of the existing syntax.
- Closure syntax that improves upon `init_scope`,
- Compatibility with normal Rust syntax & `rustfmt` support.

Upcoming Features of `pin-init`

- Many improvements coming to `pin-init` due to more usage (especially in DRM & by Danilo, many thanks!).
  - This is great, keep the suggestions coming!
- Big improvement *behind the scenes:* `syn` to parse Rust code instead of declarative macros. This allows for:
  - Better error reporting and better syntax support,
  - Comfortable maintenance (Gary Guo will be co-maintaining the library when it switches to `syn`),
  - Easier extensions of the existing syntax.
- Closure syntax that improves upon `init_scope`,
- Compatibility with normal Rust syntax & `rustfmt` support.
- More customizability for the `init!` macro (default error, should previous fields be accessible).

Language Integration

- Language integration has been started,

Language Integration

- Language integration has been started,
- Pin ergonomics make generated pin-projections obsolete,

Language Integration

- Language integration has been started,
- Pin ergonomics make generated pin-projections obsolete,
- Project goal by Alice Ryhl & Taylor Cramer on in-place initialization to implement `pin-init` as a language feature.

Language Integration

- Language integration has been started,
- Pin ergonomics make generated pin-projections obsolete,
- Project goal by Alice Ryhl & Taylor Cramer on in-place initialization to implement `pin-init` as a language feature.
  - Discussion by many different people,

Language Integration

- Language integration has been started,
- Pin ergonomics make generated pin-projections obsolete,
- Project goal by Alice Ryhl & Taylor Cramer on in-place initialization to implement `pin-init` as a language feature.
  - Discussion by many different people,
  - Many, *many* more considerations for better compatibility with existing features (`async`, `try`, classical control-flow)

Thank You for Your Attention!

Questions & Discussion